



Physics-Informed Machine Learning

Dr. André Artelt, MSc. Inaam Ashraf, MSc. Luca Hermes

{aartelt,mashraf,lhermes}@techfak.uni-bielefeld.de

WDSA CCWI 2026

Paphos, 18/05/2026



Water-Futures has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (Grant agreement No. 951424)

This research was supported by the Ministry of Culture and Science NRW (Germany) as part of the Lamarr Fellow Network. This publication reflects the views of the authors only.

Outline of Today

Time	Topics
14:00	Introduction & Welcome
	Surrogate models vs. digital twins
	Types & Implementations of surrogates
14:15	Recap of Machine Learning (ML) and Deep Learning
	Basic concepts, evaluation, hyper-parameter tuning, etc.
	Surrogate models: Challenges for ML
	Data-driven mitigation strategies (i.e., scenario generation)
14:45	Physics-informed/inspired Methods -- Part I
	Intro to Physics-informed Neural Networks (PINNs)
	Hands-On: How to build and train a PINN in Python
15:30	Coffee Break
16:00	Physics-informed/inspired Methods -- Part II
	Operators vs. PINNs
16:25	Case study: SOTA Hydraulic Surrogate Model
	Hydraulic surrogates and their applications
	Analyzing the problem and identifying relevant quantities and laws
	A SOTA Hydraulic Surrogate
	Hands-On: Usage and Evaluation of the Surrogate in Python
17:15	Closing, Summary, and Q&A
17:30	END



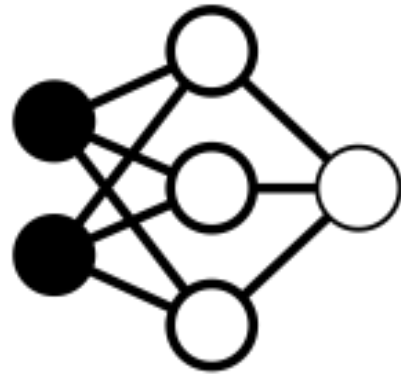
We are here!

Machine Learning - Overfitting

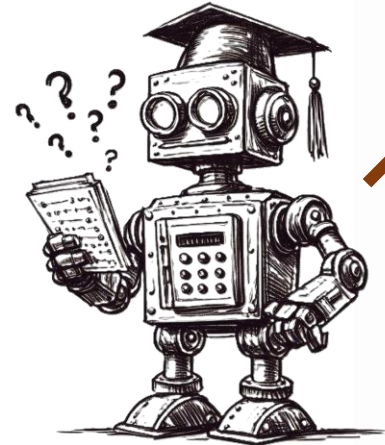


Data

+

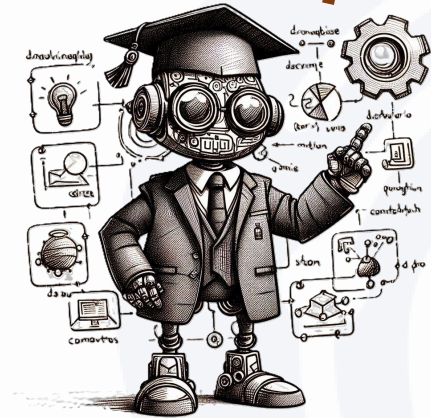
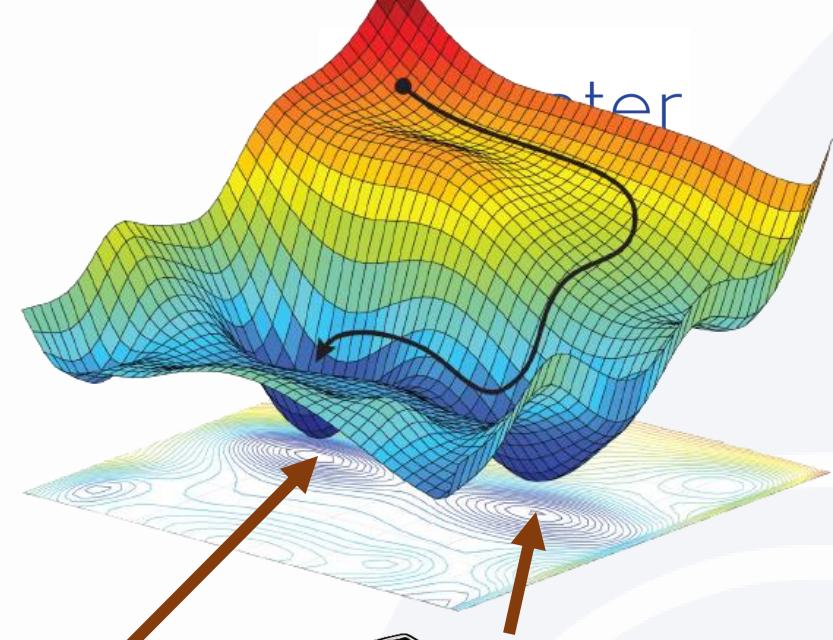


=



Solution 1

- Matches the Training data
- But not the Testing data
- Bad generalization!

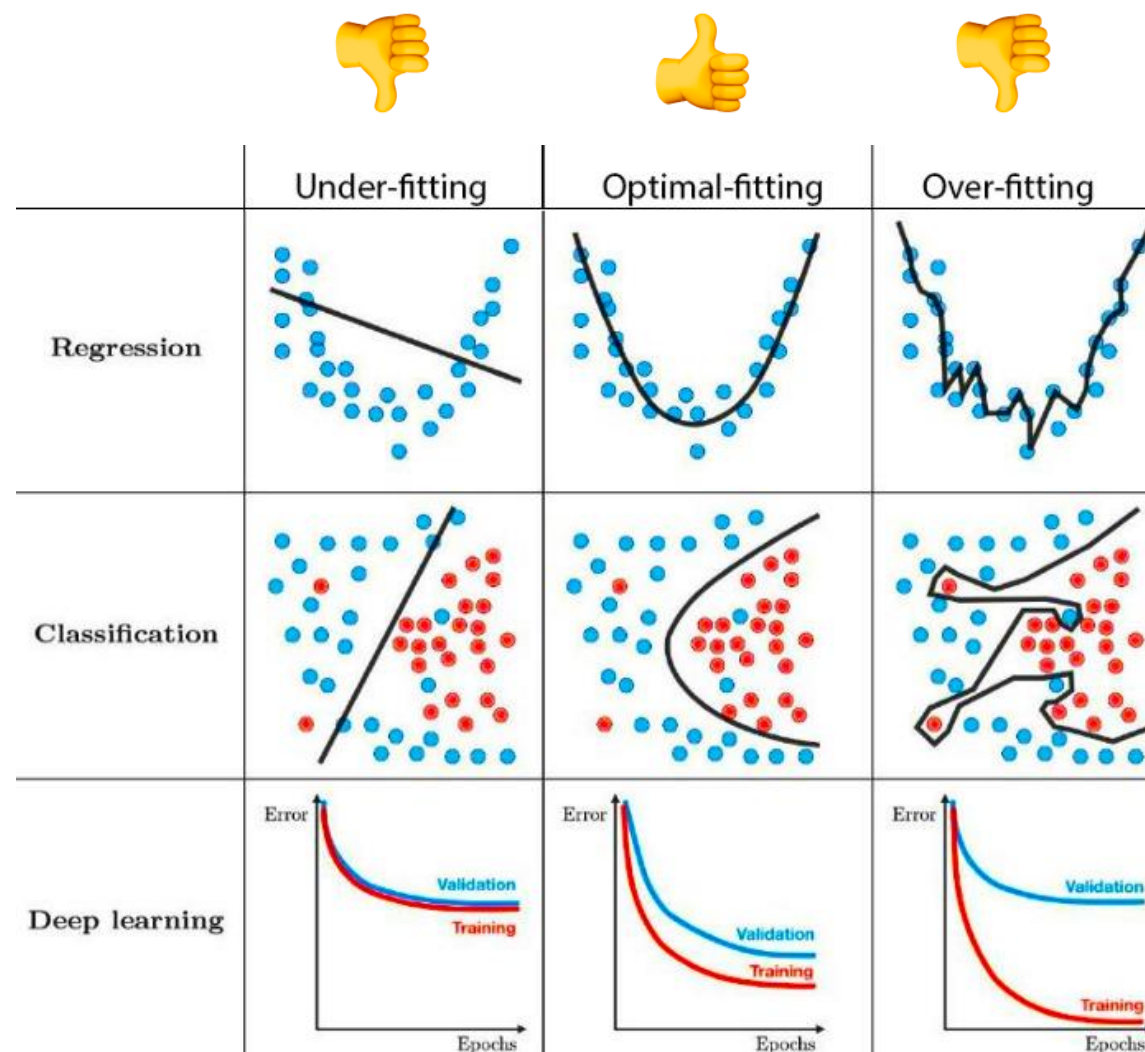


Solution 2

- Matches both Training and Testing data
- Good generalization!

Machine Learning - Overfitting

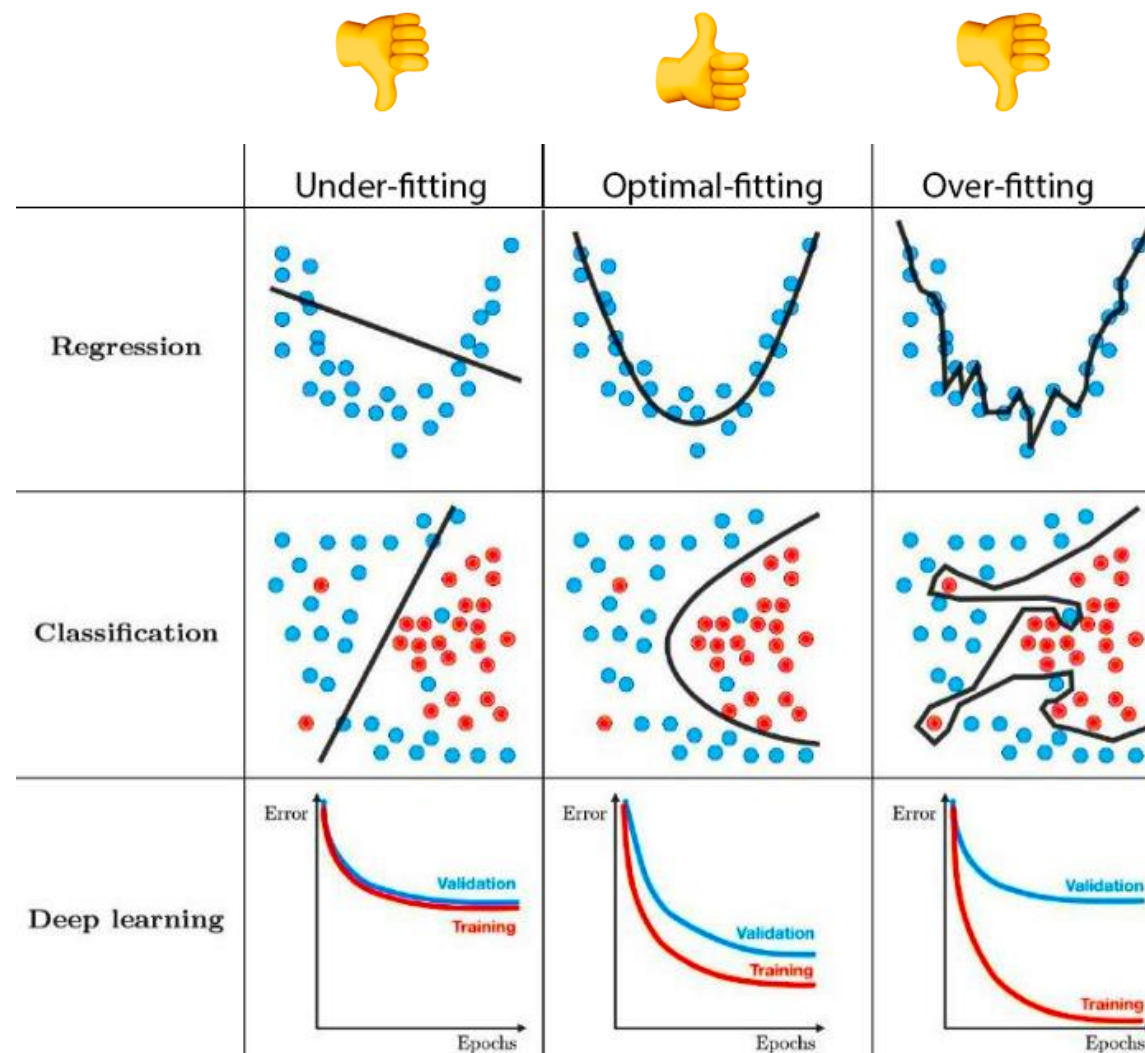
- How do we find a good fit?
 - Regularization
 - Inductive Biases
 - More or better data



Machine Learning - Overfitting

- How do we find a good fit?
 - Regularization
 - Inductive Biases
 - More or better data

Physics-Informed ML!



Inductive Bias – Use your Knowledge

- Physical Data
 - Conservation Laws / Symmetries
 - PDEs, ODEs, ...
 - Energetic Minima Preferred
 - Entropy increases
 - Verifiable even without data!

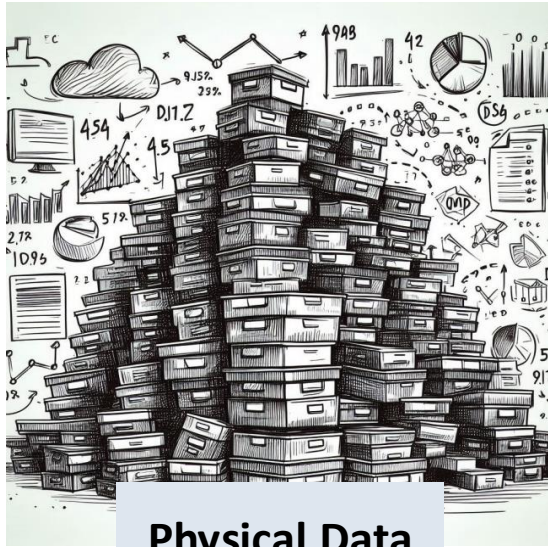
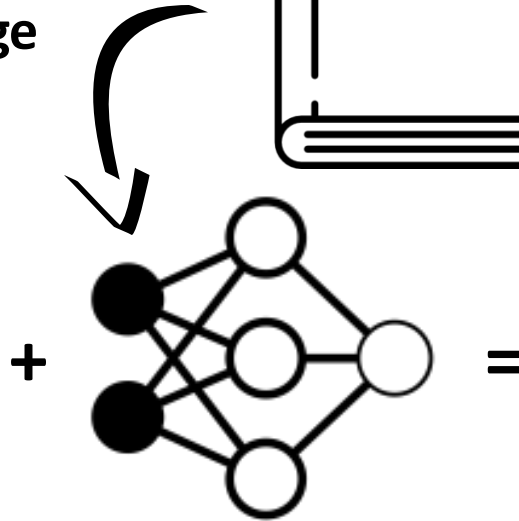
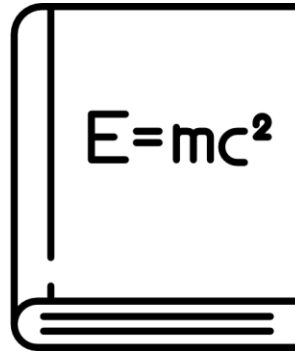


Inductive Bias – Use your Knowledge

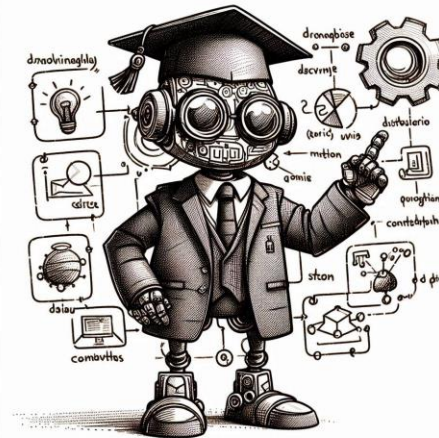
- Physical Data
 - Conservation Laws / Symmetries
 - Derivative Structure (PDEs, ODEs, ...)
 - Boundary Conditions
 - Verifiable even without data!
- Possible Realizations
 - Use additional losses - PINNs
 - Hard constrains - **Later with Inaam**
 - Choice of Basis - Later (FNOs)
 - Choice of training data

Physics-Informed Machine Learning

Physical Prior
Knowledge

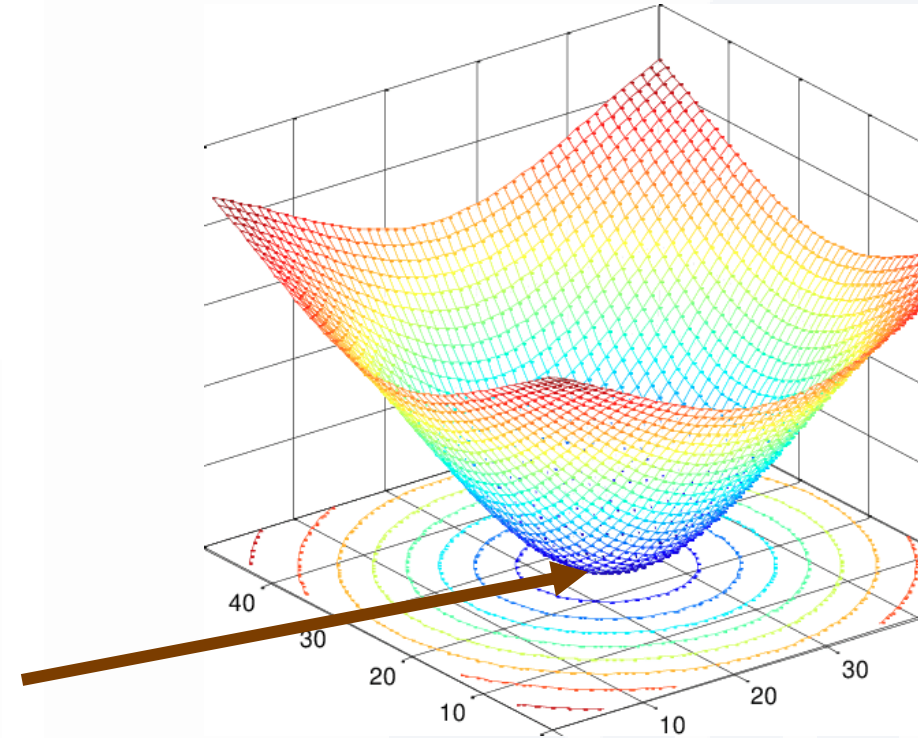


Physical Data



Solution

- Matches the data
- Obeys the Physics
- → Good generalization!



Practical Physics-Informed ML

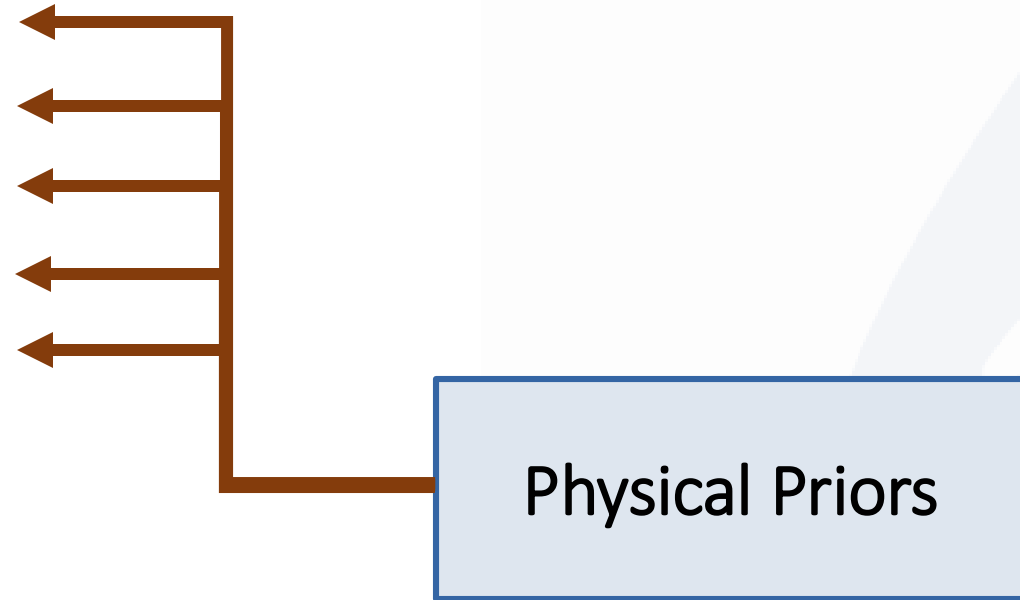
- The ML Pipeline
 1. Decide what to model
 2. Data Acquisition / Preprocessing
 3. Architecture Design
 4. Loss-shaping
 5. Training / Validation / Testing



Practical Physics-Informed ML

- The **Physics-Informed** ML Pipeline

1. Decide what to model
2. Data Acquisition / Preprocessing
3. Architecture Design
4. Loss-shaping
5. Training / Validation / Testing



Preliminary: Differential Equations

- Dynamical Systems described as partial differential equations (PDEs)
- PDEs describe how a function $u(x, t)$ changes over time $\frac{\partial u}{\partial t}$
- No closed-form solutions for $u(x, t)$

$$u(x, y, \dots, t, \alpha) : \Omega \times \mathbb{R} \times P \rightarrow \mathbb{R}$$

Diagram illustrating the function $u(x, y, \dots, t, \alpha) : \Omega \times \mathbb{R} \times P \rightarrow \mathbb{R}$ with annotations:

- Ω : Domain, e.g. a 2D plane indexed by x and y
- $\mathbb{R}$: Time
- P : PDE Parameters (e.g. diffusivity)
- $\mathbb{R}$: Function Value

Preliminary: Differential Equations

- Dynamical Systems described as partial differential equations (PDEs)
- PDEs describe how a function $u(x, t)$ changes over time $\frac{\partial u}{\partial t}$
- No closed-form solutions for $u(x, t)$
- This is heat diffusion (in 1D):

$$\begin{array}{c}
 \text{Temporal} \\ \text{change of } u
 \end{array}
 \frac{\partial u}{\partial t} =
 \begin{array}{c}
 \text{PDE Parameter:} \\ \text{Diffusivity}
 \end{array}
 D
 \begin{array}{c}
 \text{Spatial Curvature of } u \\
 \frac{\partial^2 u}{\partial x^2}
 \end{array}
 \xrightarrow{\text{Numerical Integration}}
 \begin{array}{c}
 \text{Solution} \\ \text{time } t + dt
 \end{array}
 u_{t+dt}(x) =
 \begin{array}{c}
 \text{Last Solution} \\ \text{time } t
 \end{array}
 u_t(x) +
 \begin{array}{c}
 \text{Integration} \\ \text{Step}
 \end{array}
 dt \frac{\partial u_t}{\partial t}$$

PDE Function

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}$$

Numerical
Integration

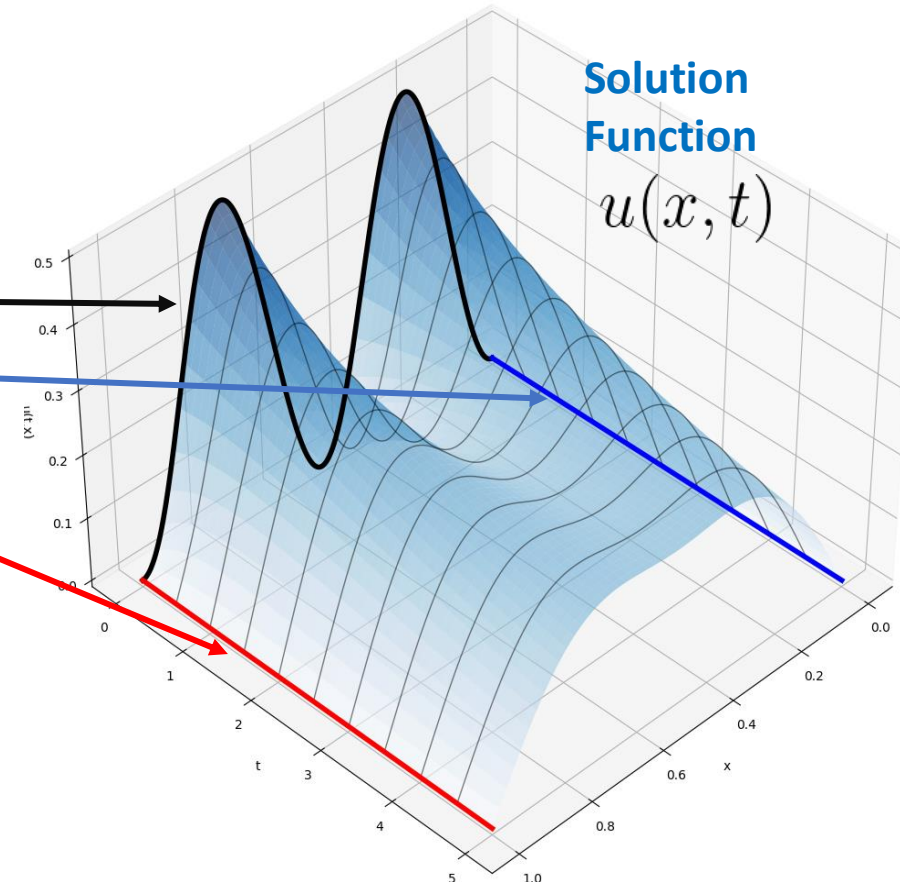
Solution Step

$$u_{t+dt}(x) = u_t(x) + dt \frac{\partial u_t}{\partial t}$$

Multi-Step
Rollout

PDE Setup:

- Parameters e.g. Diffusivity
- Initial Value
- Boundary Conditions



PDE Function

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}$$

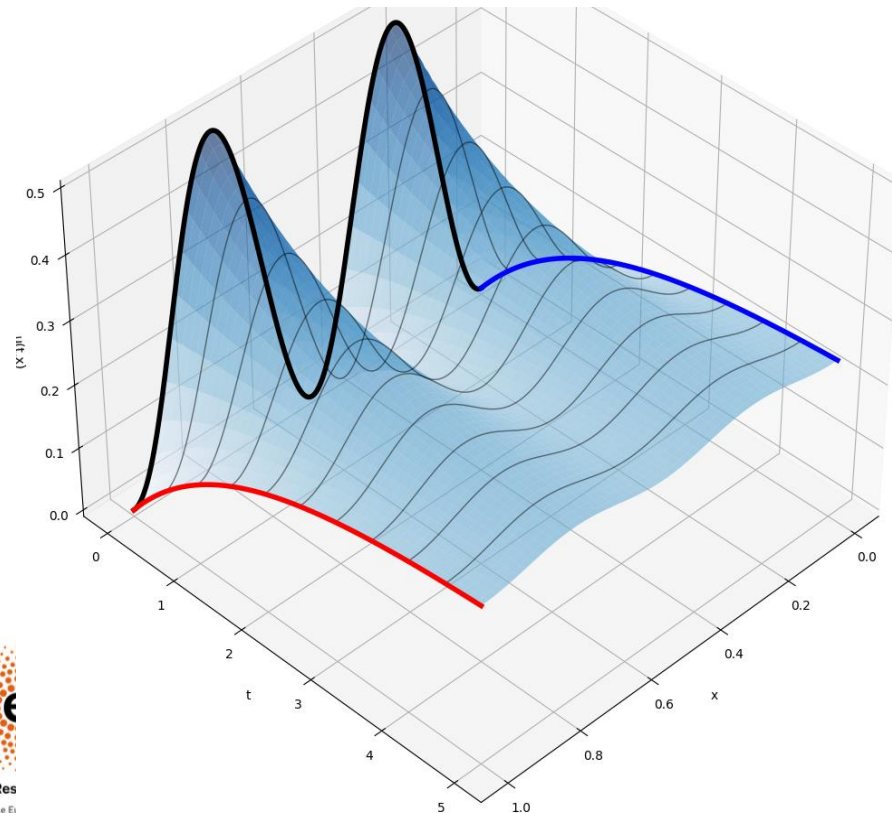


Numerical
Integration

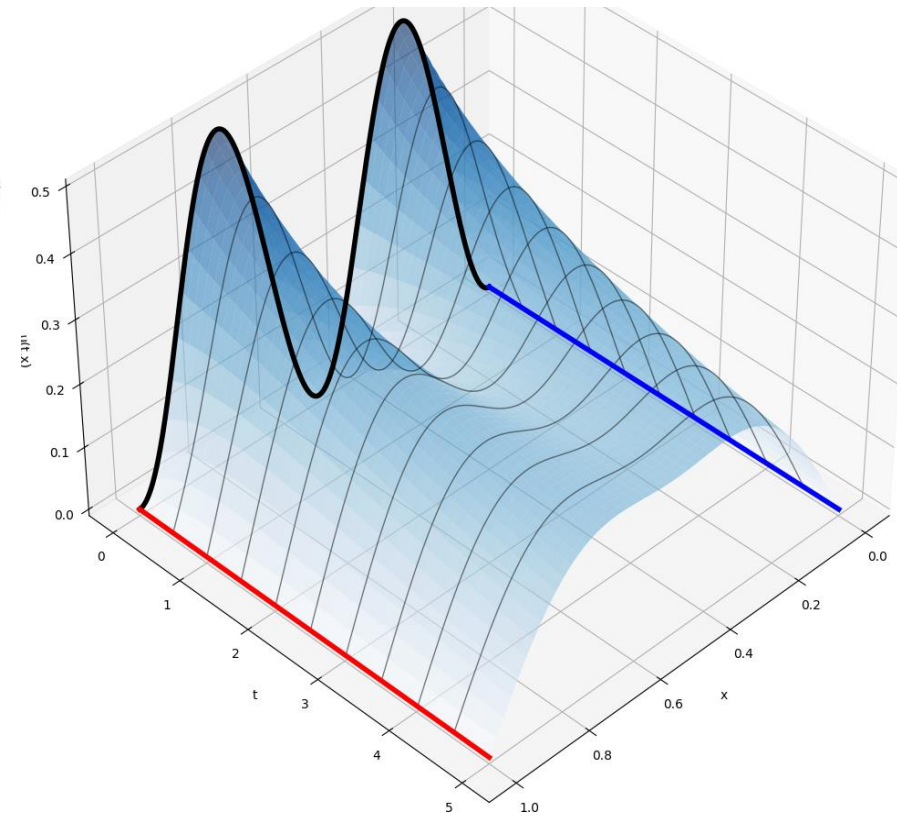
Solution Step

$$u_{t+dt}(x) = u_t(x) + dt \frac{\partial u_t}{\partial t}$$

Neumann Boundary Condition



Dirichlet Boundary Condition



Numerical
Approximation

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}$$

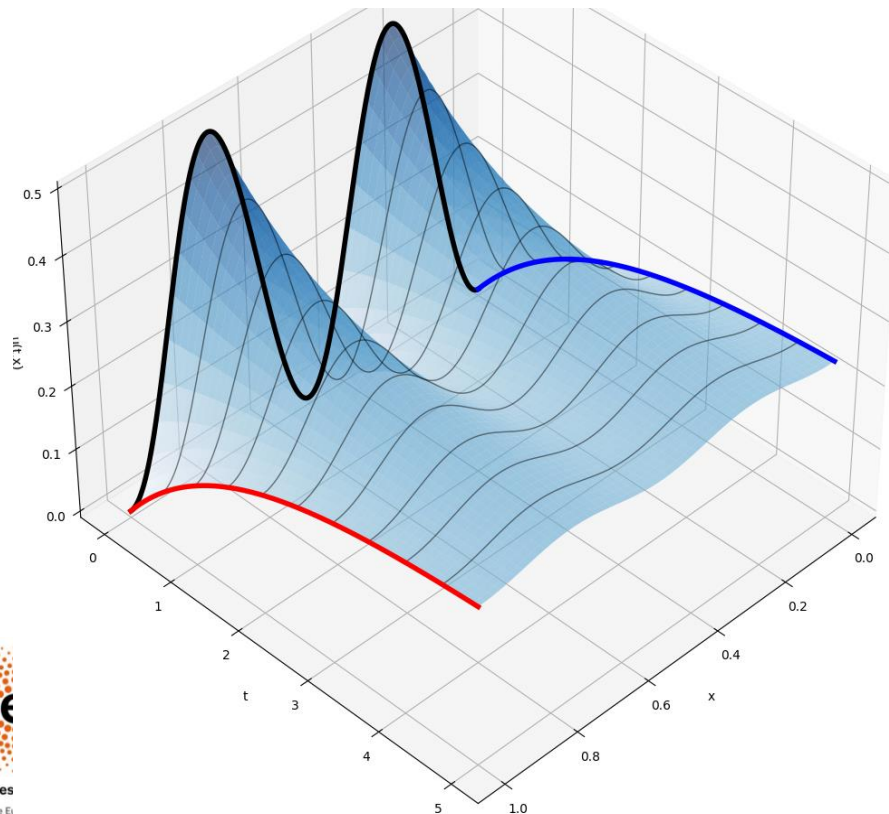


1. Stability Issues
2. Requires high resolution
3. Long Rollouts take time

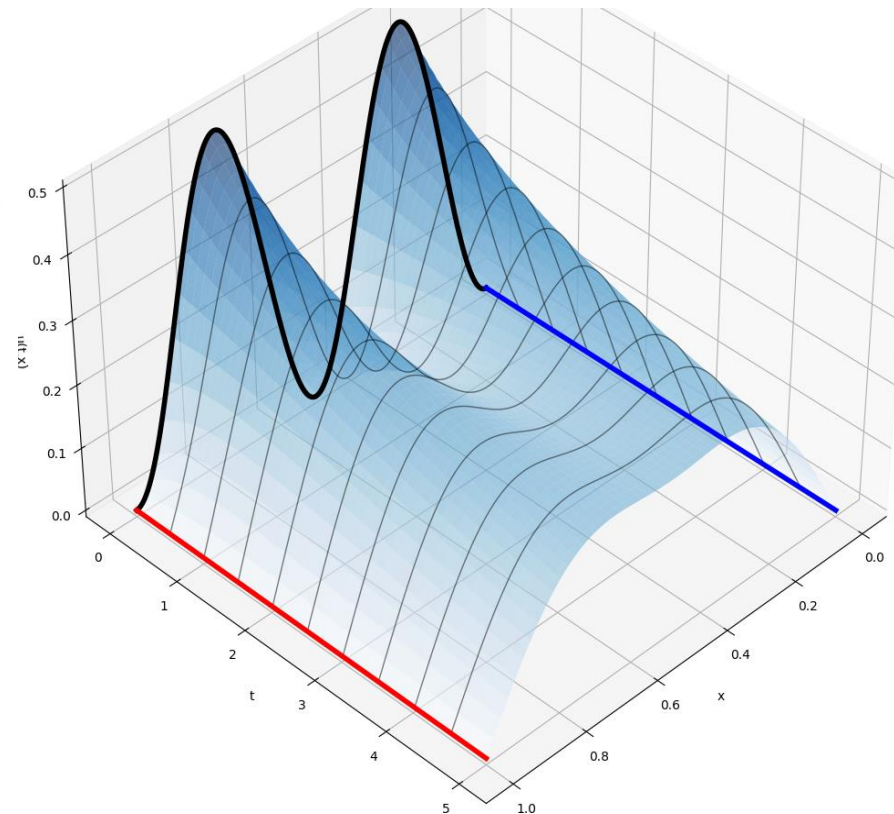


$$u_{t+dt}(x) = u_t(x) + dt \frac{\partial u_t}{\partial t}$$

Neumann Boundary Condition



Dirichlet Boundary Condition



Physics-Informed Machine Learning

Learn the Right-hand-side (PDE Operator)

- Operator NeuralODEs
- Neural ODEs
- Message-Passing Neural PDE Solvers
- PDE Net
-

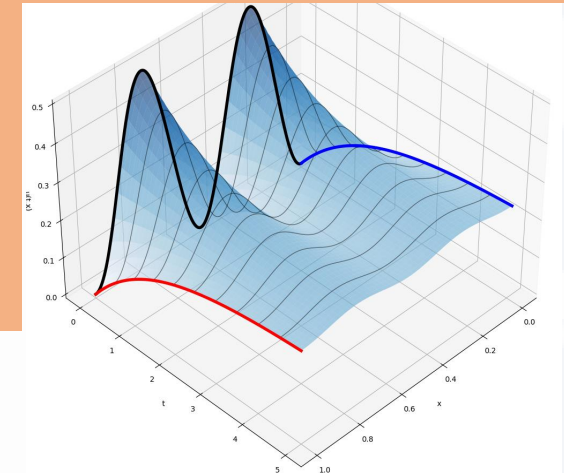
Learn the Solution Operator

- Physics-Informed Neural Networks
- DeepONets
- Fourier Neural Operator
- Laplace Neural Operators
-

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}$$

Multi-Step
Rollout

$$u(x, t)$$

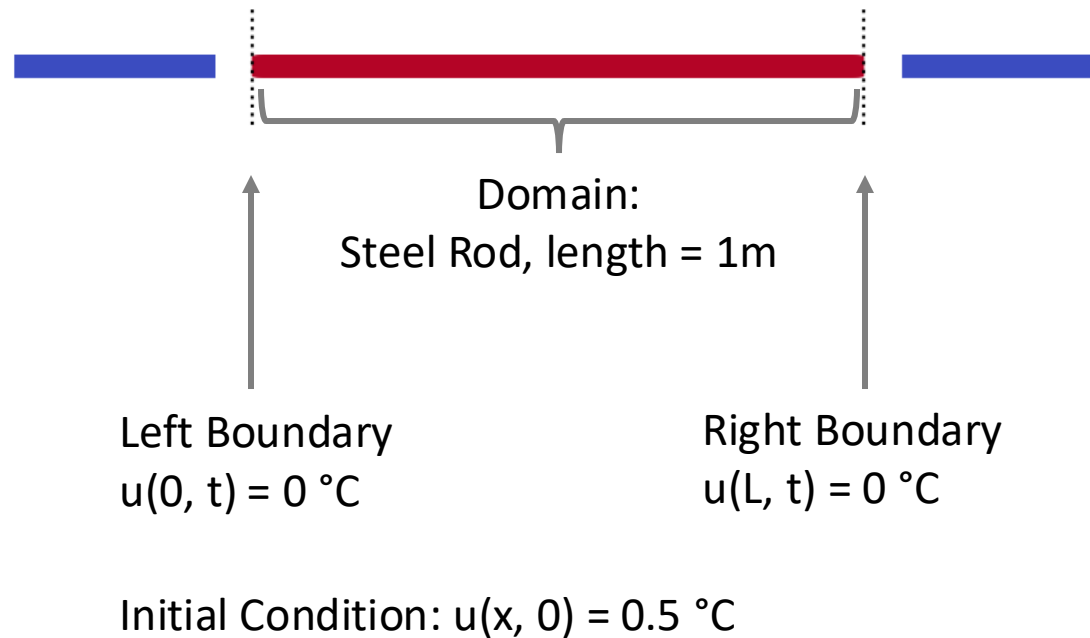


A Cornerstone: PINNs (Raissi et al. 2017)

- Example: 1D Heat Diffusion (again)
- Goal: Learning the solution function

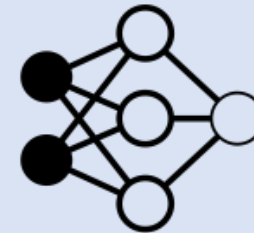
$$u(x, t) : \Omega \times \mathbb{R} \rightarrow \mathbb{R}$$

Space (1D), e.g. a metal rod Time Temperature



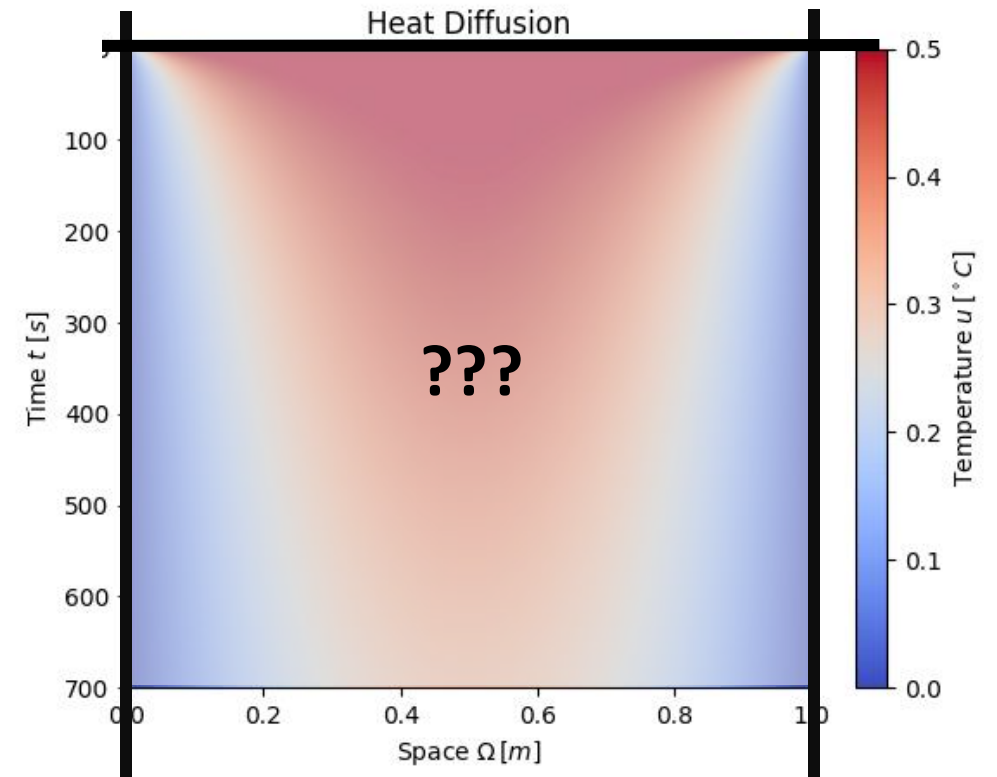
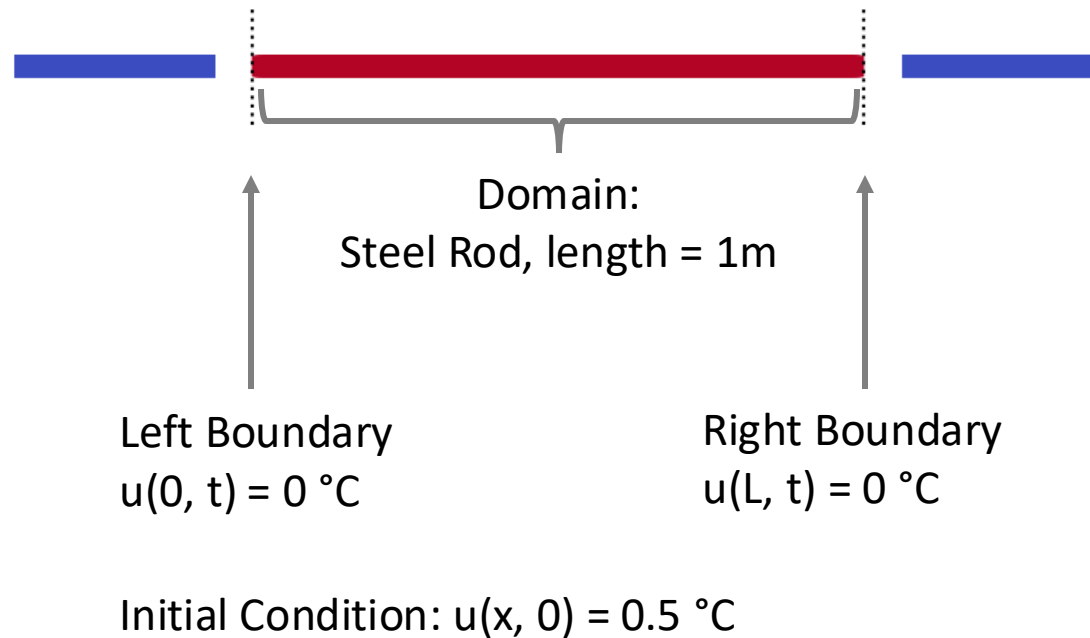
Learning Setup:

$$\underbrace{\hat{u}_\theta(x, t)} \approx u(x, t)$$



A Cornerstone: PINNs (Raissi et al. 2017)

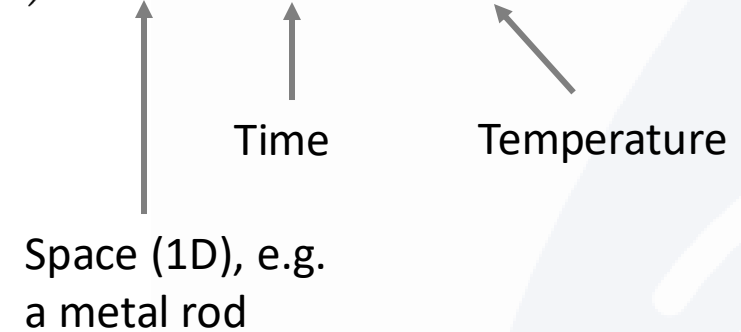
- Example: 1D Heat Diffusion (again)
- Goal: Learning the solution function $u(x, t) : \Omega \times \mathbb{R} \rightarrow \mathbb{R}$



A Cornerstone: PINNs (Raissi et al. 2017)

- **Example:** 1D Heat Diffusion (again)
- **Goal:** Learning the solution function

$$u(x, t) : \Omega \times \mathbb{R} \rightarrow \mathbb{R}$$

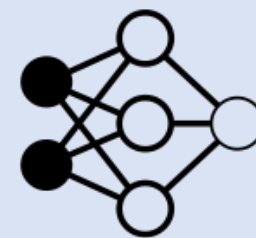


- What we know about u :
 - Initial Condition
 - Boundary Conditions
 - and

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}$$

Learning Setup:

$$\hat{u}_{\theta}(x, t) \approx u(x, t)$$



A Cornerstone: PINNs (Raissi et al. 2017)

- Neural Nets are differentiable
- Via back-propagation, we can easily compute derivatives:

$$\frac{\partial \hat{u}_\theta}{\partial t}, \frac{\partial \hat{u}_\theta}{\partial x} \quad \frac{\partial^2 \hat{u}_\theta}{\partial t^2}, \frac{\partial^2 \hat{u}_\theta}{\partial x^2}$$

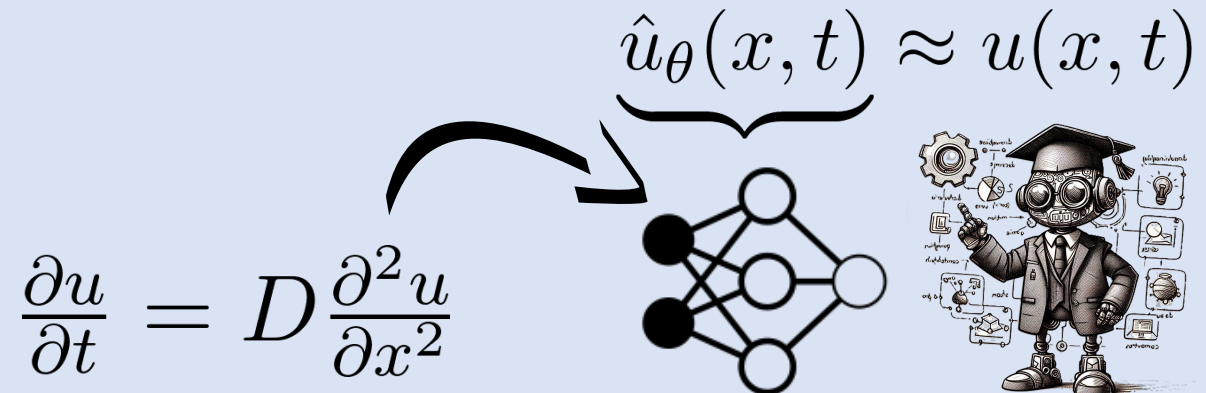
- Heat Diffusion demands:

$$\frac{\partial \hat{u}_\theta}{\partial t} - D \frac{\partial^2 \hat{u}_\theta}{\partial x^2} = 0$$

$$\mathcal{L}(x, t) = \left(\frac{\partial \hat{u}_\theta(x, t)}{\partial t} - D \frac{\partial^2 \hat{u}_\theta(x, t)}{\partial x^2} \right)^2$$

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}$$

$\hat{u}_\theta(x, t) \approx u(x, t)$



A Cornerstone: PINNs (Raissi et al. 2017)

- Initial & Boundary Conditions trained with *data MSE*:

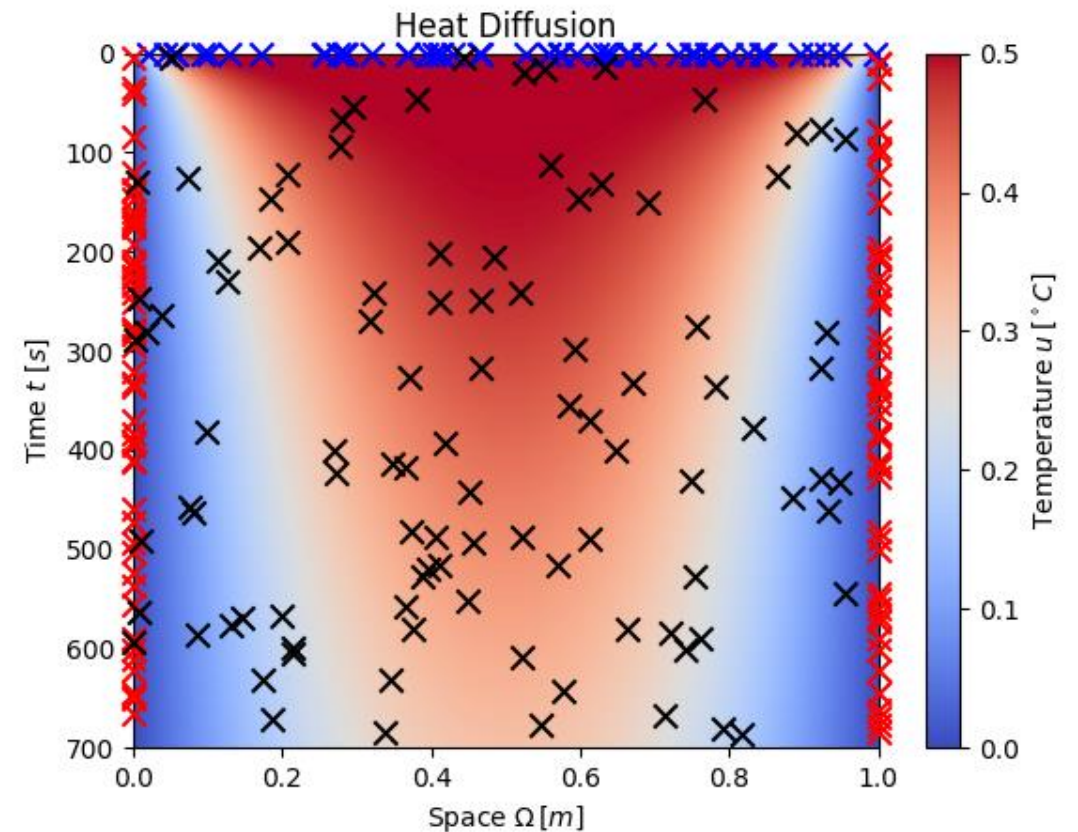
$$\mathcal{L}_{ic}(x) = (\hat{u}_{\theta}(x, 0) - 0.5)^2$$

$$\mathcal{L}_{bcl}(t) = \hat{u}_{\theta}(0, t)^2$$

$$\mathcal{L}_{bcr}(t) = \hat{u}_{\theta}(L, t)^2$$

- PDE Structure *pde MSE*

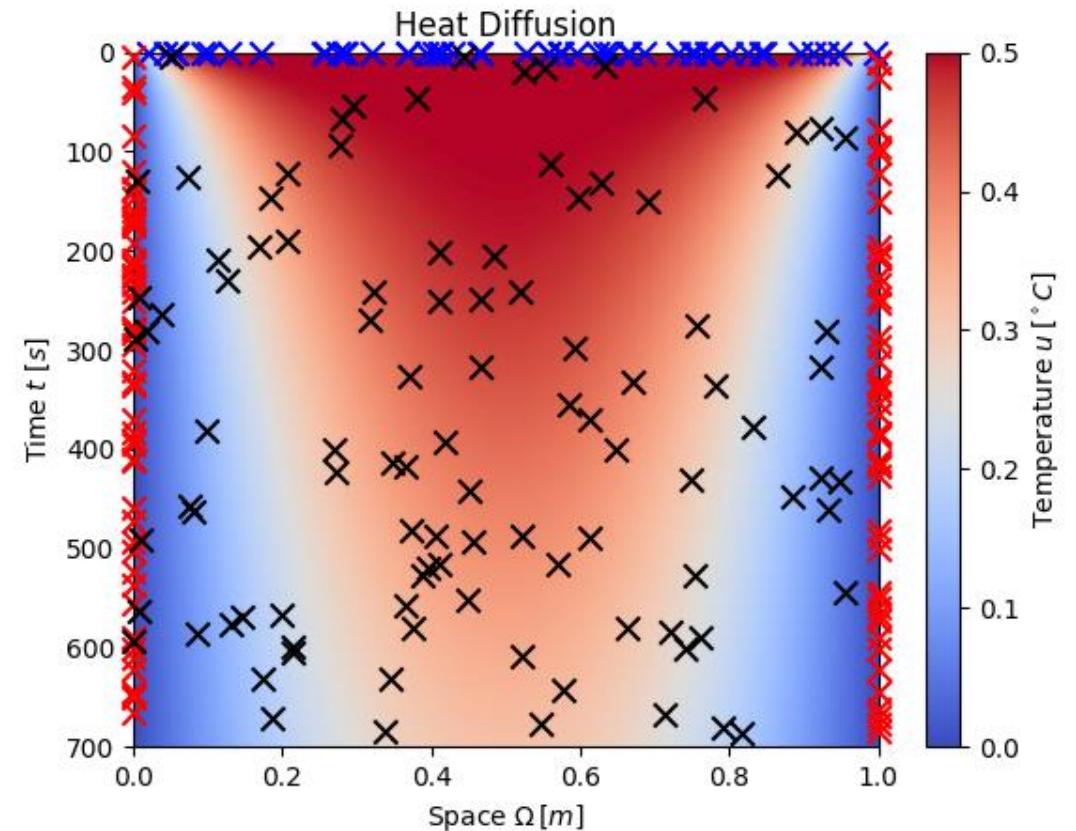
$$\mathcal{L}_{pde}(x, t) = \left(\frac{\partial \hat{u}_{\theta}(x, t)}{\partial t} - D \frac{\partial^2 \hat{u}_{\theta}(x, t)}{\partial x^2} \right)^2$$



A Cornerstone: PINNs (Raissi et al. 2017)

- Final Objective
- Optimized via Gradient Descent

$$\min_{\theta} (\mathcal{L}_{pde} + \mathcal{L}_{ic} + \mathcal{L}_{bc})$$



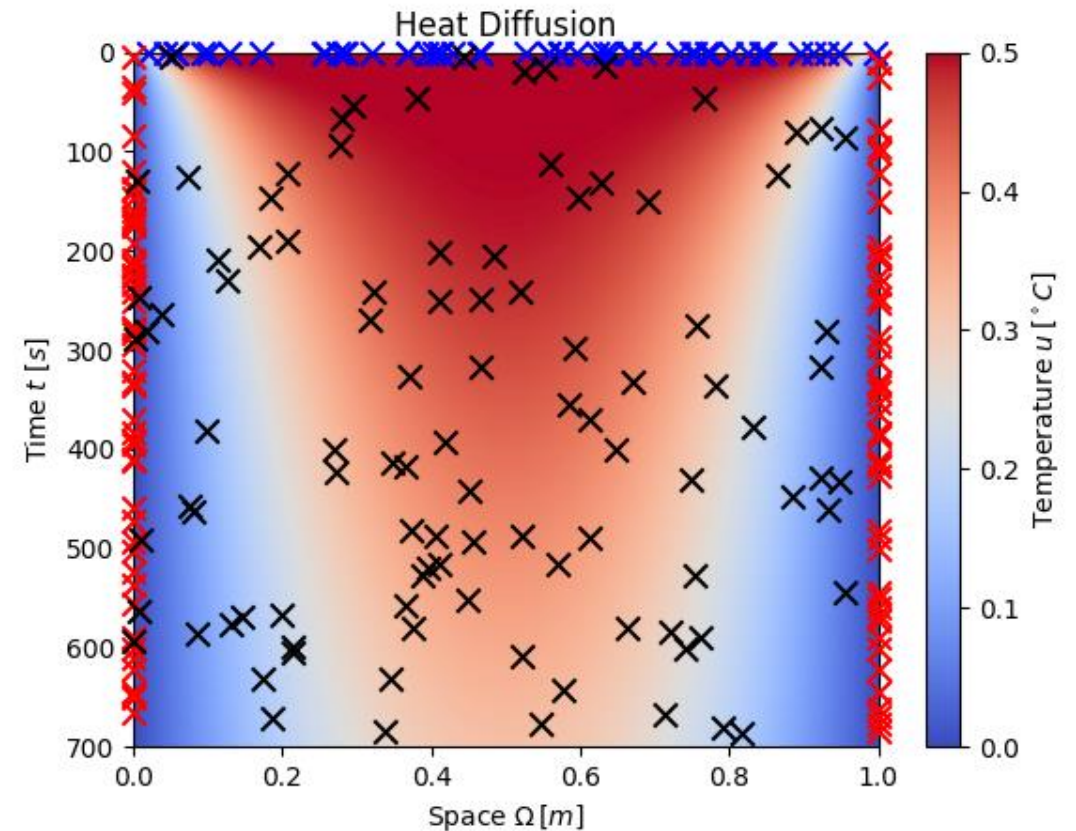
A Cornerstone: PINNs (Raissi et al. 2017)

- Final Objective
- Optimized via Gradient Descent

$$\min_{\theta} (\mathcal{L}_{pde} + \mathcal{L}_{ic} + \mathcal{L}_{bc})$$

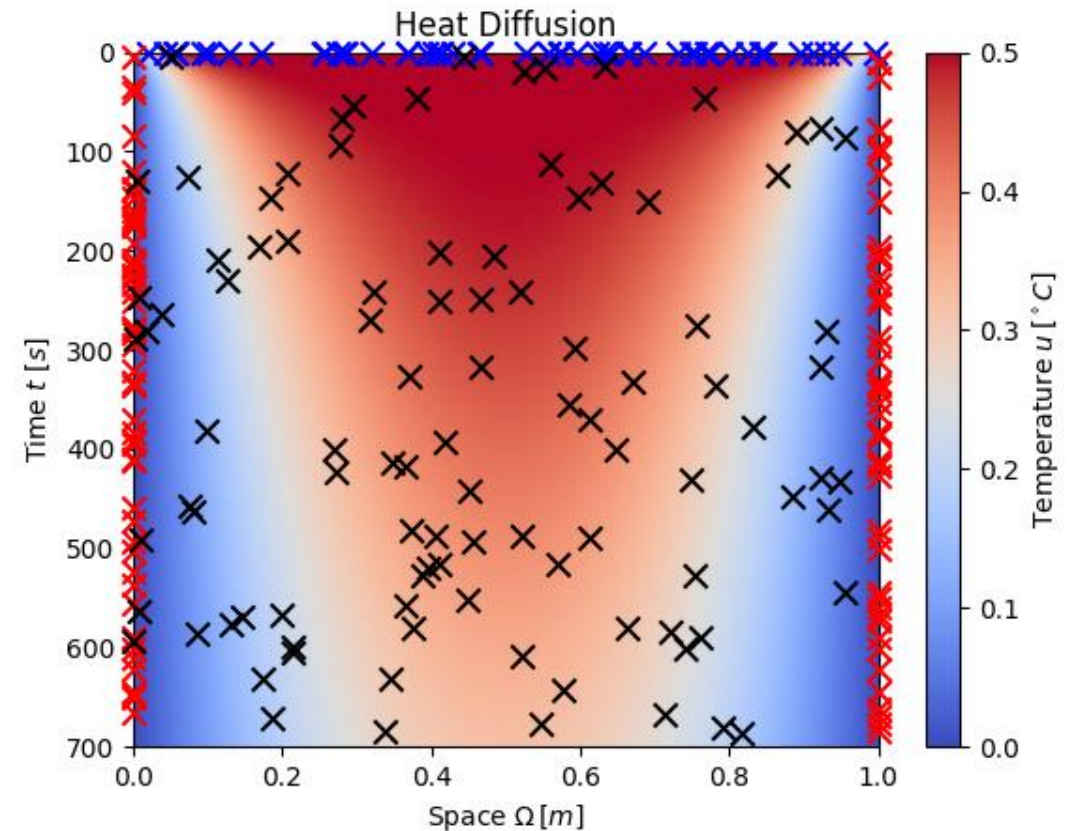
- Inverse Problems:

$$\min_{\theta, D} (\mathcal{L}_{pde} + \mathcal{L}_{ic} + \mathcal{L}_{bc})$$



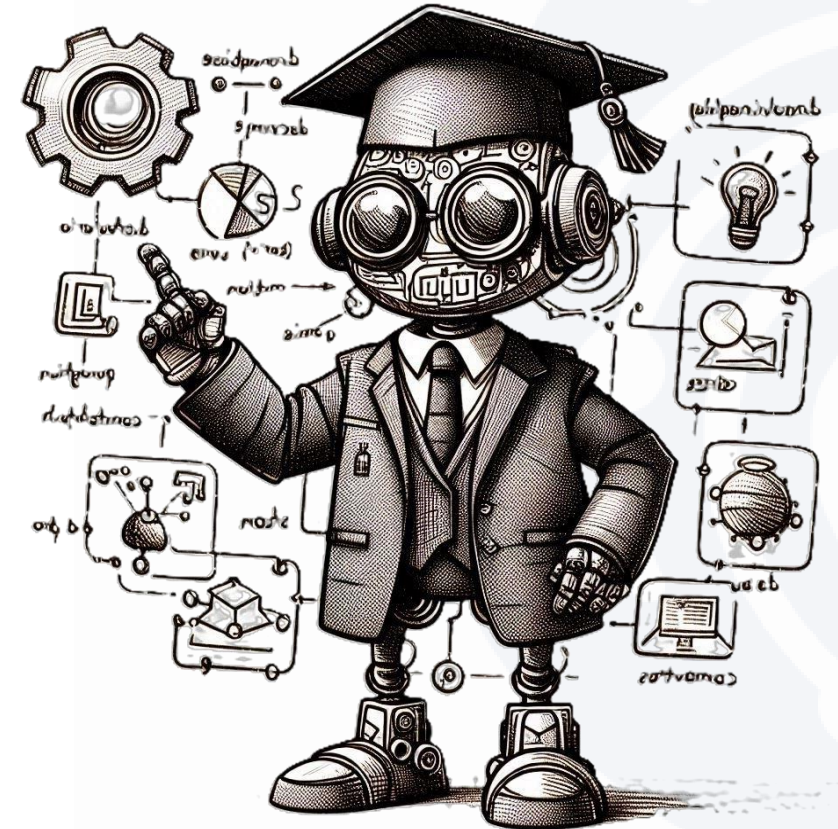
A Cornerstone: PINNs (Raissi et al. 2017)

- Strengths
 - Little to no Data required
 - Solution is continuous
 - Complicated boundary conditions can be learned from data
 - Unknown PDE Parameters can be learned (Inverse Problems)
- Weaknesses
 - Solution valid for a single PDE parametrization
 - Learned function not interpretable
 - No theoretical guarantees beyond manual validation



Key Take-Aways

- Physics-Informed ML is ...
 - ... all about finding good inductive biases
 - ... using detailed knowledge from Physics
- PINNs
 - Learn the solution function of a PDE
 - For a single Setup of a single PDE
 - Use the PDE derivative structure as a loss
 - Don't require training data
 - The rabbit hole extends:
E.g. Inverse Problems





Hands-On



Course material @GitHub
<https://tinyurl.com/2m4vy8vp>